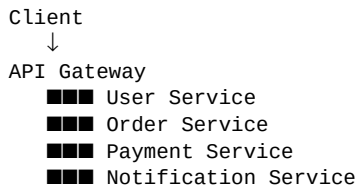


API Gateway — Practical System Design Notes

Hinglish • Routing • Auth • Rate Limiting • Resilience • Observability • Production Architecture

1. API Gateway Kya Hai?

API Gateway system ka controlled entry point hai. Client directly har internal service ko call karne ke bajay Gateway ko request bhejta hai. Gateway authentication, routing, rate limiting, validation, logging aur kuch cross-cutting concerns handle kar sakta hai.



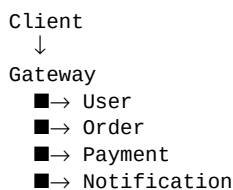
2. Gateway Ki Need Kyun?

Agar 10 microservices hain aur client directly sabko call kare, to client ko service URLs, authentication rules, versions aur failure behavior ka knowledge rakhna padega. Gateway ek controlled boundary provide karta hai.

Without Gateway:

```
Client → User
Client → Order
Client → Payment
Client → Notification
```

With Gateway:



3. API Gateway Responsibilities

- Routing: /users → User Service, /orders → Order Service.
- Authentication/JWT validation.
- Authorization checks where appropriate.
- Rate limiting.
- Request validation.
- TLS termination.
- API version routing.
- Timeouts and controlled retries.
- Logging, metrics and tracing.
- Request/response transformation when needed.
- Caching for suitable APIs.

4. Reverse Proxy vs API Gateway

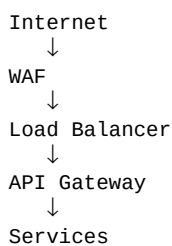
Reverse Proxy	API Gateway
---------------	-------------

Forwards traffic to backend	API-aware entry layer
Routing/load distribution	Routing + auth + policies
Nginx is common example	Kong/Envoy/API Gateway products are common
Can do TLS/load balancing	Can add rate limits, API keys, transformations, observability

5. Load Balancer vs API Gateway

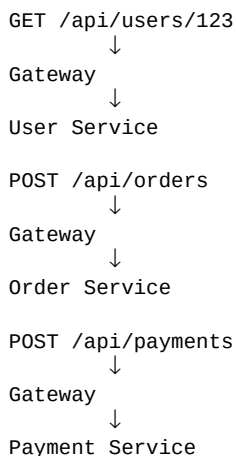
Load Balancer	API Gateway
Mainly distributes traffic	Controls and manages API traffic
L4/L7 balancing	API routes/policies
Health checks	Auth/rate-limit/validation + routing
Often behind CDN/WAF	Often sits at API boundary

6. WAF vs Gateway



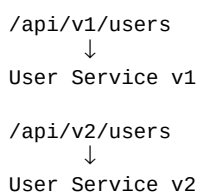
WAF ka focus malicious/web attack traffic filtering hai. Gateway ka focus API traffic management hai. Real architecture mein dono complementary ho sakte hain.

7. Routing



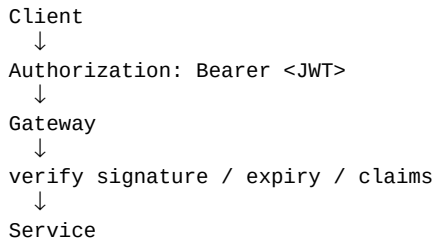
Routing rules path, HTTP method, host, headers ya version ke basis par ho sakte hain.

8. API Versioning



Versioning backward compatibility manage karne mein help karta hai. Gateway old and new versions ko different upstreams/services tak route kar sakta hai.

9. Authentication at Gateway



Gateway JWT signature, expiry aur basic claims validate kar sakta hai. Lekin business-level authorization har case mein Gateway par shift karna zaroori nahi; service ko bhi resource authorization enforce karna chahiye.

10. JWT Validation

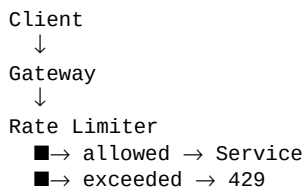
- Signature valid?
- Token expired?
- Issuer/audience correct?
- Required claims present?

Invalid token par generally request service tak nahi jaati.

11. API Key

Public/developer APIs mein API keys client/application identification aur quota management ke liye use ho sakti hain. API key ko user authentication ka universal replacement mat samjho.

12. Rate Limiting



Gateway edge par basic protection dena useful hai. Distributed limits ke liye Redis jaisa shared state store use kiya ja sakta hai.

13. Rate Limit Dimensions

- IP
- User ID
- API key
- Tenant
- Endpoint
- Combination: tenant + endpoint

14. 429 Too Many Requests

HTTP 429 Too Many Requests

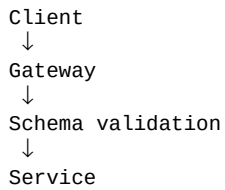
Retry-After: 10

Meaning:

Client ne allowed request rate exceed kar di.

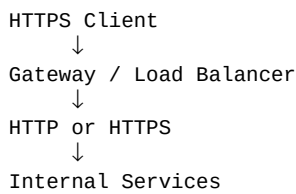
Client ko controlled retry behavior dena chahiye. Random aggressive retry gateway/service ko aur overload kar sakta hai.

15. Request Validation



Basic schema validation gateway par duplicate bad traffic ko services tak pahunchne se rok sakti hai. Business validation service ke paas rehni chahiye.

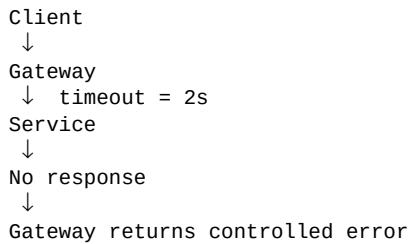
16. TLS Termination



TLS termination edge par karne se certificate management centralized ho sakta hai. Highly sensitive environments mein internal encryption bhi required ho sakta hai.

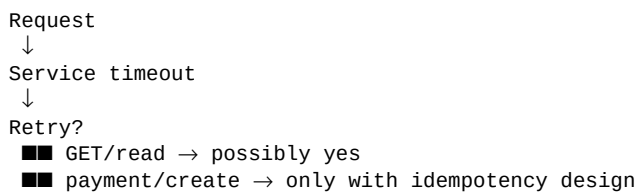
17. Timeout

Har upstream request ke liye bounded timeout rakho. Unlimited waiting connections consume karega aur cascading failure create kar sakta hai.



18. Retry

Retry sirf safe/idempotent operations ke liye carefully use karo. POST payment jaise operations ko blindly retry karna duplicate side effects create kar sakta hai.



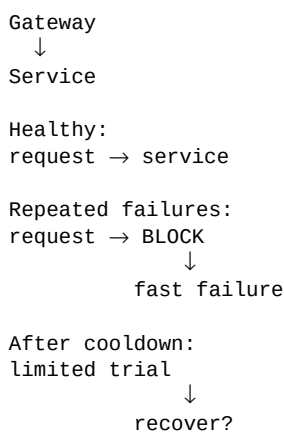
19. Exponential Backoff + Jitter

Retry 1 → ~100ms
Retry 2 → ~200ms
Retry 3 → ~400ms

+ random jitter

Jitter synchronized retry bursts ko reduce karta hai.

20. Circuit Breaker



Circuit breaker failing dependency ko continuously hammer hone se protect karta hai.

21. Circuit Breaker States

State	Meaning
Closed	Requests normally pass
Open	Requests fail fast; dependency protected
Half-Open	Limited test requests check recovery

22. Cascading Failure

Service A
↓
Service B is slow
↓
A connections pile up
↓
Gateway waits
↓
more requests pile up
↓
system becomes overloaded

Timeouts, bounded concurrency, circuit breakers, load shedding and rate limits cascading failures ko control karne ke tools hain.

23. Request Correlation ID

Client
↓
X-Request-ID: abc123
↓
Gateway
↓
User Service
↓
Order Service

Logs:
abc123 → same request

Distributed debugging mein correlation/request ID bahut useful hai.

24. Distributed Tracing

```
Client
↓
Gateway span
↓
Order span
↓
Payment span
↓
DB span
```

Tracing se pata chal sakta hai total latency kis service/dependency mein consume hui.

25. Gateway Logging

- Request ID.
- Route.
- HTTP method.
- Status code.
- Latency.
- Upstream service.
- Rate-limit result.
- Error category.

Sensitive data jaise passwords, tokens ya payment secrets ko blindly logs mein mat daalo.

26. Gateway Metrics

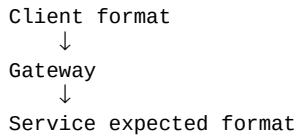
- Requests/sec.
- p50/p95/p99 latency.
- 4xx rate.
- 5xx rate.
- 429 rate.
- Upstream timeout rate.
- Circuit breaker open count.
- Rate-limit rejection count.
- Per-route traffic.
- Gateway CPU/memory/connections.

27. Gateway Caching

```
Client
↓
Gateway
↓
Cache
■→ HIT → response
■→ MISS → service → cache
```

Read-heavy, cacheable APIs ke liye useful. Personalized/private responses mein cache key and authorization carefully design karo.

28. Request Transformation



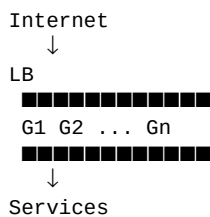
Gateway headers, paths ya payload transformations kar sakta hai, but excessive business logic Gateway mein rakhne se Gateway complex bottleneck ban sakta hai.

29. Service Discovery

Dynamic environments mein Gateway ko service instances ka current location discover karna pad sakta hai. Kubernetes environments mein service DNS, service mesh ya platform-native discovery common patterns hain.

30. Gateway as a Bottleneck

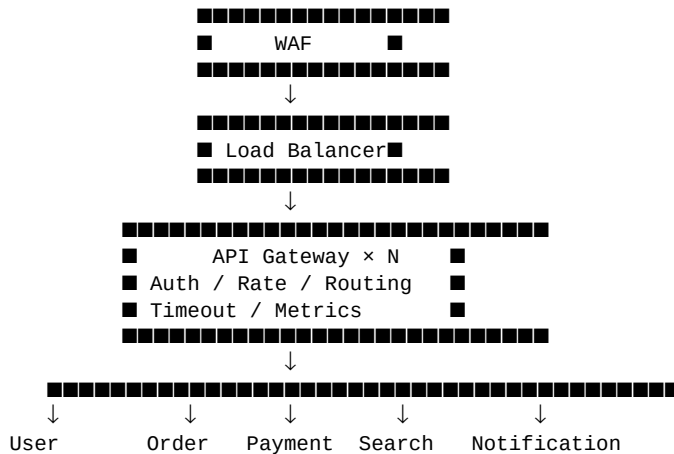
Gateway khud single instance nahi hona chahiye. Multiple replicas + load balancing + stateless design rakho.



31. Stateless Gateway

Gateway instance ke local memory mein critical shared session/rate-limit state mat rakho. Shared state Redis/database/service layer mein ho, ya gateway ko horizontally scalable architecture mein design karo.

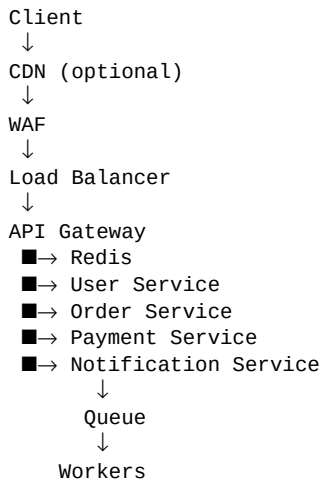
32. Production Architecture



Shared:

Redis / Queue / DB / Observability

33. Gateway + Existing System Components



Gateway ke andar business services ka code nahi hona chahiye. Gateway primarily traffic-management boundary hai.

34. Failure Scenarios

Problem	Typical response
Gateway instance down	LB sends traffic to healthy replicas
Service slow	Timeout + possibly circuit breaker
Too many requests	Rate limit / 429
Service unavailable	Fast failure/fallback where valid
Redis unavailable	Defined fail-open/fail-closed policy depending on feature
Gateway overloaded	Horizontal scale + load balancing + limits
Bad deployment	Rollback/canary strategy
Auth provider slow	Timeout/cache/public policy as appropriate

35. What Should NOT Live in Gateway?

- Complex business workflows.
- Heavy database operations.

- Long-running background jobs.
- Payment business state machine.
- Large file processing.
- Deep domain authorization logic that belongs to the service.

Gateway ko thin rakho: routing + cross-cutting controls + observability.

36. Node.js API Gateway — Simple Mental Model

Express Gateway

```
app.use(authMiddleware)
app.use(rateLimitMiddleware)

app.use('/users', proxy(userService))
app.use('/orders', proxy(orderService))
app.use('/payments', proxy(paymentService))
```

Production mein mature gateway/proxy tooling use karna operationally simpler ho sakta hai; custom Node gateway tab useful hai jab aapko application-specific control chahiye.

37. Local Hands-On Project

Node.js + Express ke saath 3 backend services aur ek gateway banao.

Client

↓

Gateway : 3000

■→ User Service : 3001

■→ Order Service : 3002

■→ Payment Service : 3003

Redis : 6379

- Gateway routing implement karo.
- JWT authentication middleware add karo.
- Redis-based rate limiter add karo.
- Request ID generate/forward karo.
- Timeout add karo.
- One service intentionally slow karo.
- Circuit breaker behavior test karo.
- Prometheus metrics expose karo.
- Grafana mein p95 latency aur 5xx/429 rate dekho.
- 3 gateway instances run karke LB behavior test karo.

38. Interview Scenario: Payment API

Interviewer: Gateway payment request ko retry karega?

Answer: Blind retry nahi. Payment create operation side effects create karta hai. Idempotency key + payment state machine ke saath controlled retry/status recovery design karunga.

39. Interview Scenario: Service Slow Hai

Answer: Upstream timeout set karunga, metrics/tracing se latency identify karunga, repeated failures par circuit breaker use karunga, aur retry sirf safe operations par bounded backoff+jitter ke saath karunga.

40. Interview Scenario: Gateway Down

Answer: Gateway ko horizontally replicated/stateless deploy karunga aur upstream LB ke through multiple healthy instances rakhunga. Health checks aur deployment rollback/canary strategy use karunga.

41. Interview Q&A;

Question	Short answer
API Gateway kya?	API traffic ka controlled entry point for routing and cross-cutting policies.
LB vs Gateway?	LB distributes traffic; Gateway adds API-aware policies and routing.
WAF vs Gateway?	WAF security filtering; Gateway API traffic management.
JWT gateway par?	Signature/expiry/basic claims validate kar sakta hai.
Rate limiting?	Requests ko defined identity/route limits ke according control karna.
429?	Rate limit exceeded response.
Timeout kyun?	Slow dependency ko resources consume karte rehne se rokne ke liye.
Retry kab?	Safe/idempotent operations; bounded backoff+jitter.
Circuit breaker?	Failing dependency ko repeated calls se protect karta hai.
Gateway stateless kyun?	Easy horizontal scaling and failover.
Gateway bottleneck?	Replicas, LB, efficient policies and observability.
Business logic gateway mein?	Minimum; domain logic service mein.

42. Final Cheat Sheet

API GATEWAY

CLIENT



WAF



LOAD BALANCER



API GATEWAY × N

- Authentication
- Authorization boundary
- Routing
- Rate limiting
- Validation
- TLS termination
- Timeout
- Retry (careful)
- Circuit breaker
- Caching
- Request ID
- Metrics/tracing



MICROSERVICES

Golden interview line:

"I'll keep the API Gateway stateless and horizontally scalable. It will handle routing, authentication, rate limiting, timeouts, observability and other cross-cutting concerns, while keeping business logic inside the respective services. For resilience, I'll use bounded timeouts, carefully scoped retries and circuit breakers, and I'll monitor p95/p99 latency, 4xx/5xx, 429s and upstream failures."